



Алгоритмы обработки больших данных

L07. CEP, CDC, ML, Metrics, Failure recovery,
Execution configuration, Async IO, Delivery
guarantees. Best (worst) practices

Апрель, 2026

Смирнов Сергей

smirnov.work@mail.ru

[@serg_v_smirnov](#)

Agenda

1. BigData intro
2. Stream & flink intro
3. Batch mode, pyflink runtime architecture
4. DataStream operators API, keyed state
5. Stateful stream processing. Windows, Triggers, ProcessFunction, State, Timers, Watermarks
6. State and Chekpoint, State Backends. Table API и SQL API
- 7. CEP, CDC, ML, Metrics, Failure recovery, Execution configuration, Async IO, Delivery guarantees. Best (worst) practices**
8. Flink-SQL, базовый синтаксис, работа с окнами
9. Flink-SQL, соединение потоков, udf, оптимизация
10. Whats new, trends (fluss, streamhouse)

Заключительный проект



Написать генератор сообщений (раз в секунду) от IoT устройств вида:

- Тип устройства (id, идентификатор)
- Время события
- Температура
- Влажность

Генератор сообщений IoT

Эти события публикуем в топик кафки

kafka

Разработать ddl dml скрипты для создания и наполнения справочника типов IoT устройств

- Id идентификатор типа
- TypeName наименование типа

pg

ddl.sql

dml.sql

src: kafka

src: pg

Рассчитываем среднюю температуру и медиану влажности в каждой минуте

join

window

sink: kafka

Соединяем события из кафки со статичным справочником из постгри

flink, работа в event time

Сохраняем полученный результат в топик кафки в формате

- Время (hh:mm)
- Тип устройства (из pg)
- Средняя температура
- Медиана влажности

kafka

- Реализовать источник (source) и или получатель (sink) на sql/table api
- Реализовать переход между datastream и sql/table api

CEP

A B+ C* D? E

```
Pattern<Car, ?> start2010 = Pattern.<Car>begin("start").where(new
SimpleCondition<>() {
    @Override
    public boolean filter(Car value){
        return value.year > 2010;
    }
});

Pattern<Car, ?> nameA = start2010.next("next").where(new
SimpleCondition<Car>() {
    @Override
    public boolean filter(Car car) throws Exception {
        return car.model.startsWith("A");
    }
});
```

```
tenv.sql_query("SELECT c.amodel, c.ayear2, c.bmodel, c.byear2 \n" +
    "FROM cars\n"+
    "MATCH_RECOGNIZE (\n" +
    "ORDER BY ts2\n" +
    "MEASURES\n" +
    "A.model as amodel, \n" +
    "A.year2 as ayear2, \n" +
    "B.model as bmodel, \n" +
    "B.year2 as byear2 \n" +
    "ONE ROW PER MATCH\n" +
    "AFTER MATCH SKIP PAST LAST ROW\n" +
    "PATTERN (A B)\n" +
    "DEFINE\n" +
    "A as A.year2 > 2010,\n"+
    "B as B.model like 'A%'\n"+
    ") AS c"
).execute().print();
```

<https://nightlies.apache.org/flink/flink-docs-stable/docs/libs/cep/>

https://nightlies.apache.org/flink/flink-docs-stable/docs/dev/table/sql/queries/match_recognize/

CDC

```
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import com.ververica.cdc.debezium.JsonDebeziumDeserializationSchema;
import com.ververica.cdc.connectors.mysql.source.MySqlSource;
```

```
public class MySqlBinlogSourceExample {
    public static void main(String[] args) throws Exception {
        MySqlSource<String> mySqlSource = MySqlSource.<String>builder()
            .hostname("yourHostname")
            .port(yourPort)
            .databaseList("yourDatabaseName") // set captured database
            .tableList("yourDatabaseName.yourTableName") // set captured table
            .username("yourUsername")
            .password("yourPassword")
            .deserializer(new JsonDebeziumDeserializationSchema()) // converts
SourceRecord to JSON String
            .build();
```

```
    StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();
```

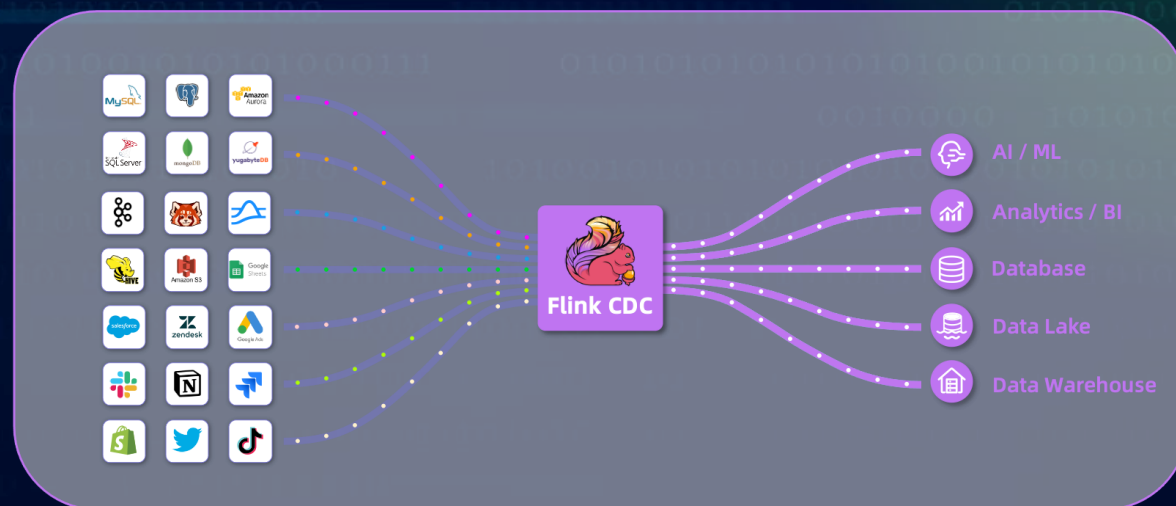
```
    env.enableCheckpointing(3000);
```

```
    env.fromSource(mySqlSource, WatermarkStrategy.noWatermarks(), "MySQL")
        // set 4 parallel source tasks
        .setParallelism(4)
        .print().setParallelism(1); // use parallelism 1 for sink to keep message ordering
```

```
    env.execute("Print MySQL Snapshot + Binlog");
```

```
}
}
```

<https://nightlies.apache.org/flink/flink-cdc-docs-stable/>



CDC Connectors for Apache Flink® integrates Debezium as the engine to capture data changes

<https://github.com/debezium/debezium>

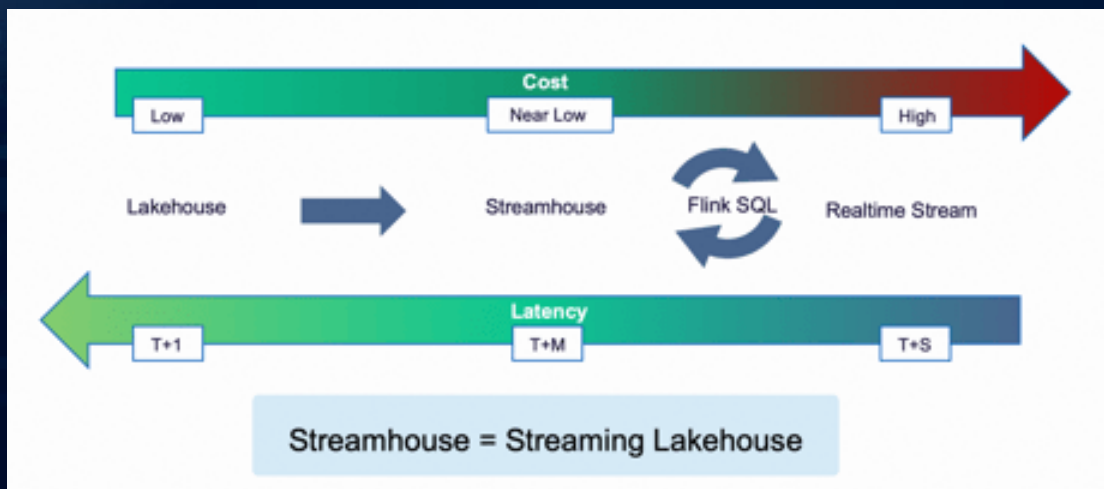
-- creates a mysql cdc table source

```
CREATE TABLE mysql_binlog ( id NOT NULL, name STRING, description
STRING, weight DECIMAL(10,3), PRIMARY KEY(id) NOT ENFORCED)
WITH ('connector' = 'mysql-cdc', 'hostname' = 'localhost',
'port' = '3306', 'username' = 'flinkuser',
'password' = 'flinkpw', 'database-name' = 'inventory',
'table-name' = 'products' );
```

-- read snapshot and binlog data from mysql, do some transformation, show on the client

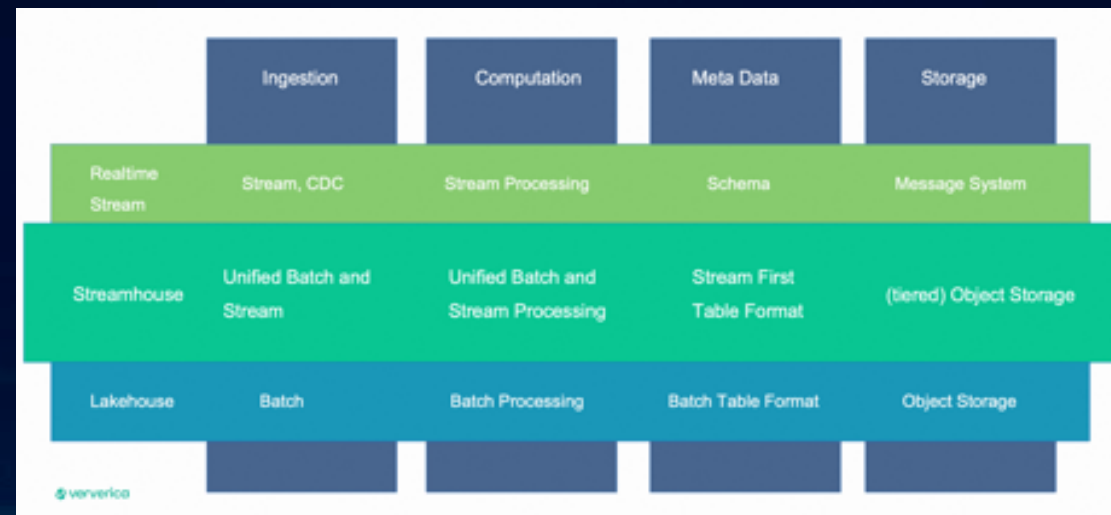
```
SELECT id, UPPER(name), description, weight FROM mysql_binlog;
```

Streamhouse

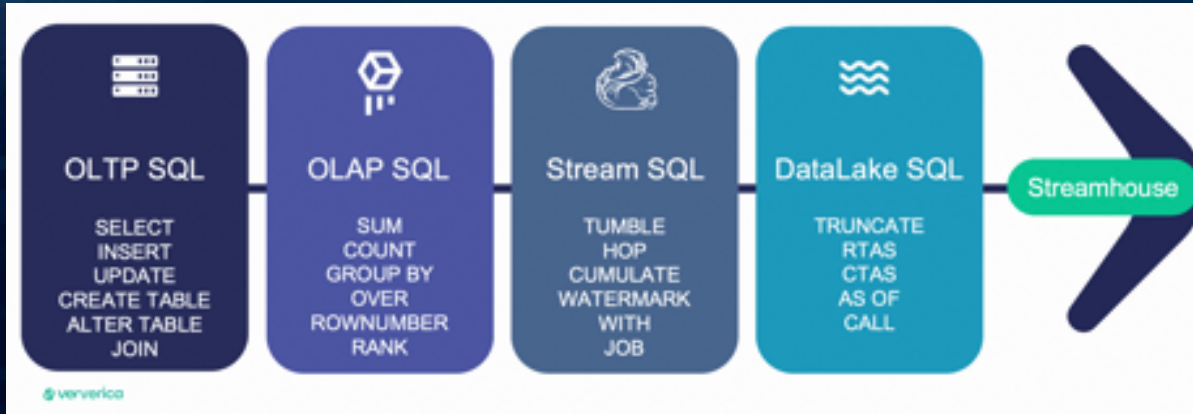


Significant improvement in latency from days to minutes outweighs the increased cost compared to Lakehouse

Streamhouse integrates aspects of both approaches. It processes data in both batch and streaming modes, with the table format aligning with streaming concepts

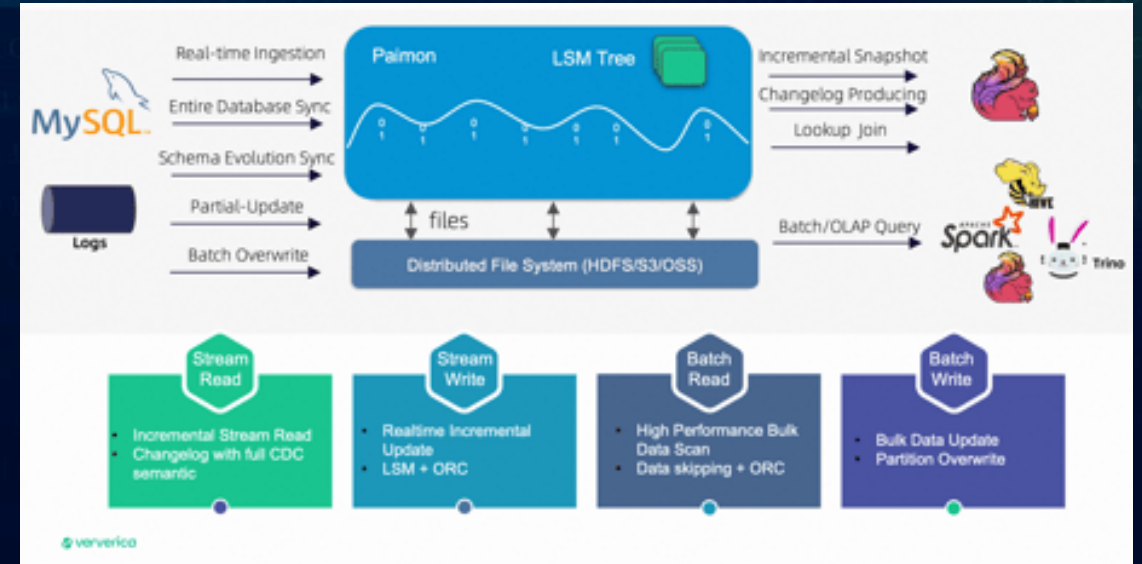


Streamhouse, flink + paimon



Flink already offers an extensive set of syntax, including DDL, DML, aggregation, joins, and more. Yet the full potential of Flink SQL is often underestimated. It can be categorized into four main types:

- **OLTP SQL**: for traditional data manipulation and querying.
- **OLAP SQL**: for data warehouse ETL processes.
- **Stream SQL**: for unique streaming data processing logic, such as tumble or hop windows.
- **Data Lake SQL**: for executing common lake house tasks, like creating a table as select.



Architecture positions Paimon as a unified batch and stream data lake platform, boasting various exciting features:

- Adept support for both batch read/write and stream read/write operations.
- Provision of scalable metadata by leveraging cost-effective data lake storage.
- Enhanced versatility, through the use of rich merge engines, such as deduplication (retraining the last entry), partial update, and aggregation.
- Simplified construction of streaming pipelines through Changelog producers, offering various patterns like 'input,' 'lookup,' and 'full-compaction.' This ensures easy creation and management of changelogs for your data.

<https://paimon.apache.org/docs/master/concepts/overview/>

<https://www.ververica.com/blog/streamhouse-unveiled>

ML

<https://nightlies.apache.org/flink/flink-ml-docs-master/docs/try-flink-ml/java/build-your-own-project/>

AI era

```
// 1. Set up the local environment
EnvironmentSettings settings = EnvironmentSettings.inStreamingMode();
TableEnvironment tEnv = TableEnvironment.create(settings);

// 2. Create a source table from in-memory data
Table myTable = tEnv.fromValues(
    ROW(FIELD("text", STRING())),
    row("Hello"),
    row("Machine Learning"),
    row("Good morning")
);

// 3. Create model
tEnv.createModel(
    "my_model",
    ModelDescriptor.forProvider("openai")
        .inputSchema(Schema.newBuilder().column("input", STRING()).build())
        .outputSchema(Schema.newBuilder().column("output", STRING()).build())
        .option("endpoint", "https://api.openai.com/v1/chat/completions")
        .option("model", "gpt-4.1")
        .option("system-prompt", "translate to chinese")
        .option("api-key", "<your-openai-api-key-here>")
        .build()
);

Model model = tEnv.fromModel("my_model");

// 4. Use the model to make predictions
Table predictResult = model.predict(myTable, ColumnList.of("text"));

// 5. Async prediction example
Table asyncPredictResult = model.predict(
    myTable,
    ColumnList.of("text"),
    Map.of("async", "true")
);
```

Open AI

```
CREATE MODEL ai_analyze_sentiment
INPUT (`input` STRING)
OUTPUT (`content` STRING)
WITH (
  'provider'='openai',
  'endpoint'='https://api.openai.com/v1/chat/completions',
  'api-key' = '<YOUR KEY>',
  'model'='gpt-3.5-turbo',
  'system-prompt' = 'Classify the text below into one of the following labels: [positive, negative, neutral, mixe
);
```

```
CREATE TEMPORARY VIEW movie_comment(id, movie_name, user_comment, actual_label)
AS VALUES
(1, 'Good Stuff', 'The part where children guess the sounds is my favorite. It's a very romantic narrative compar
```

```
INSERT INTO print_sink
SELECT id, movie_name, content as predicat_label, actual_label
FROM ML_PREDICT(
  TABLE movie_comment,
  MODEL ai_analyze_sentiment,
  DESCRIPTOR(user_comment));
```

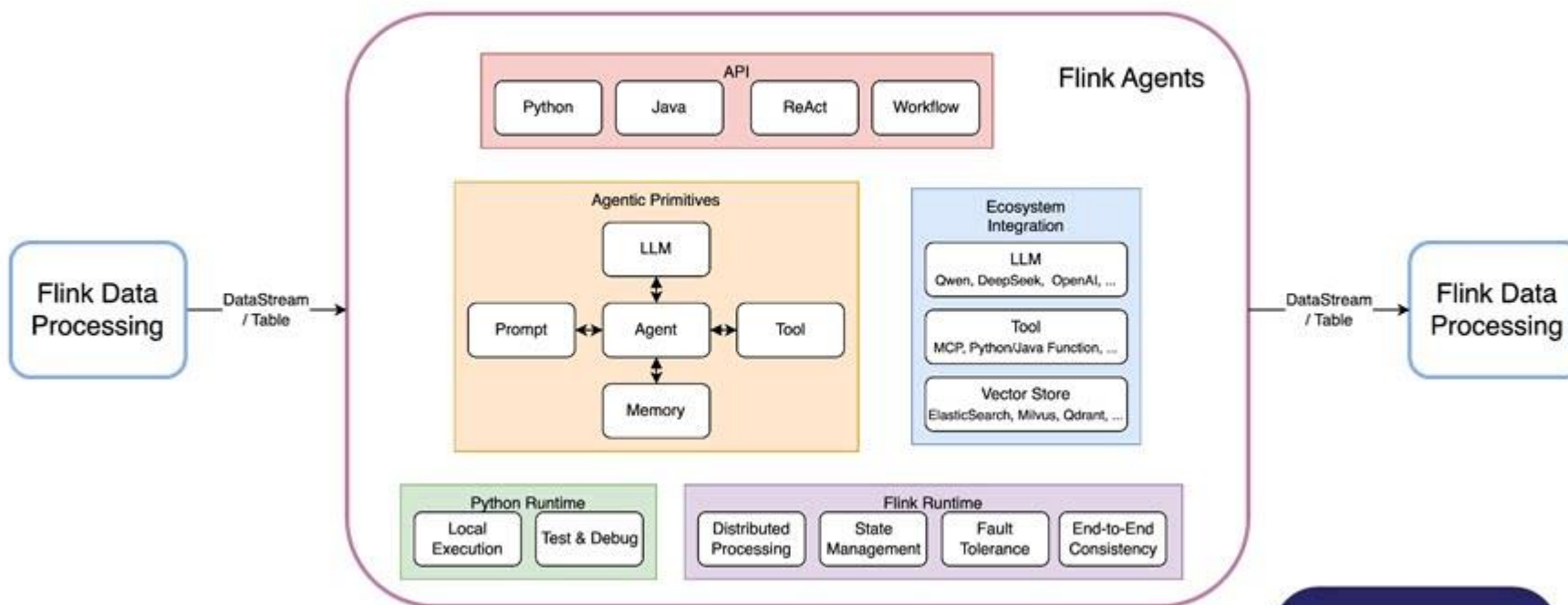
Open AI

```
from pyflink.table import TableEnvironment, EnvironmentSettings, DataTypes, Schema
from pyflink.table.model_descriptor import ModelDescriptor
env_settings = EnvironmentSettings.in_streaming_mode()
t_env = TableEnvironment.create(env_settings)
t_env.get_config().set("pipeline.jars", "file:///home/serega/SD/jars/flink-model-openai-2.2.0.jar")
t_env.execute_sql("""
    CREATE TEMPORARY VIEW input_table (id, review_text) AS VALUES
      (1, 'Hi, there'),
      (2, 'Tell smth about python flink')
""")
in_schema = Schema.new_builder() \
    .column("review_text", DataTypes.STRING()) \
    .build()
out_schema = Schema.new_builder() \
    .column("prediction", DataTypes.STRING()) \
    .build()
model_desc = ModelDescriptor.for_provider("openai") \
    .input_schema(in_schema) \
    .output_schema(out_schema) \
    .option("api-key", "your-key") \
    .option("endpoint", "https://api.openai.com/v1/chat/completions") \
    .option("model", "gpt-4o") \
    .option("system-prompt", "Analyze the sentiment of the following text.") \
    .build()
t_env.create_temporary_model("sentiment_model", model_desc)
result_table = t_env.execute_sql("""
    SELECT
      id,
      review_text,
      prediction
    FROM ML_PREDICT(
      TABLE input_table,
      MODEL sentiment_model,
      DESCRIPTOR(review_text)
    )
""")
result_table.print()
```

Not tested:
com.openai.errors.PermissionDeniedException: 403: Country, region, or territory not supported

flink-agents

Architecture



COMMUNITY
THE ASF CONFERENCE
CODE

graph

Not supported since 1.17

<https://nightlies.apache.org/flink/flink-docs-release-1.16/docs/libs/gelly/overview/>

Configuration

```
./bin/flink run-application -t yarn-application \  
-Djobmanager.memory.process.size=1024m \  
-Dtaskmanager.memory.process.size=1024m \
```

Parameters

```
"$FLINK_DIR"/bin/kubernetes-session.sh -Dkubernetes.cluster-  
id=${CLUSTER_ID} \  
-Dkubernetes.container.image.ref=${FLINK_IMAGE_NAME} \  
-Djobmanager.memory.process.size=1088m \  
-Dkubernetes.jobmanager.cpu=0.5 \  
-Dkubernetes.taskmanager.cpu=0.5 \
```

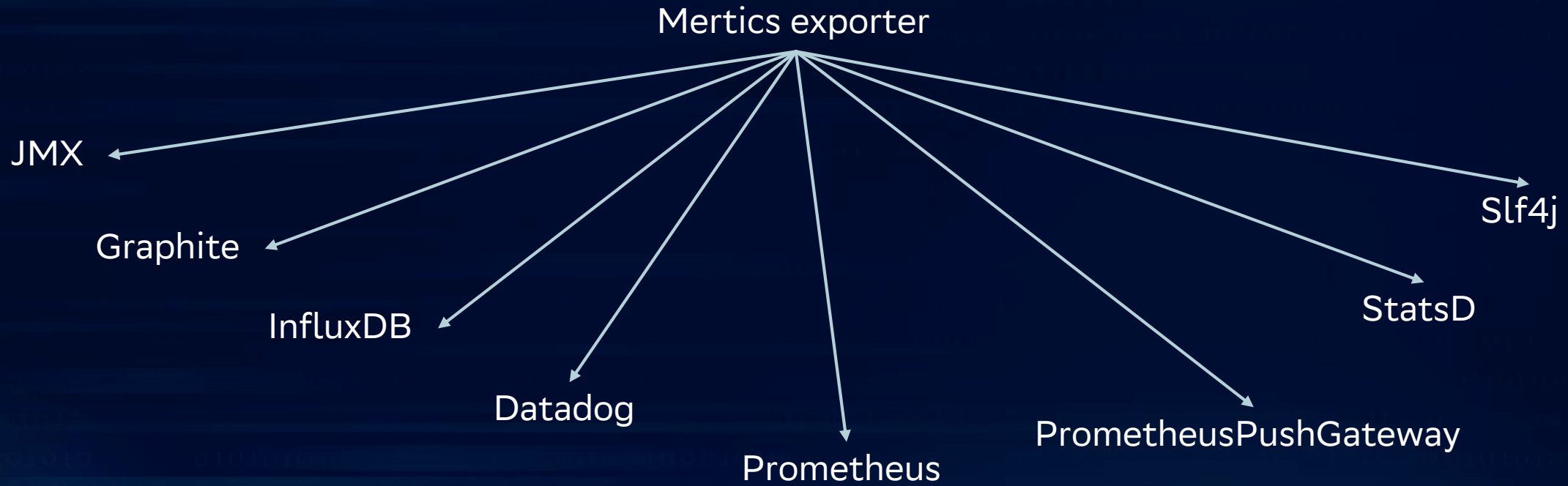
```
flink-conf.yaml  
piVersion: v1  
kind: ConfigMap  
metadata:  
  name: flink-config  
  labels:  
    app: flink  
data:  
  flink-conf.yaml: |+  
    jobmanager.rpc.address: flink-jobmanager  
    taskmanager.numberOfTaskSlots: 2  
    blob.server.port: 6124
```

flink-conf.yaml

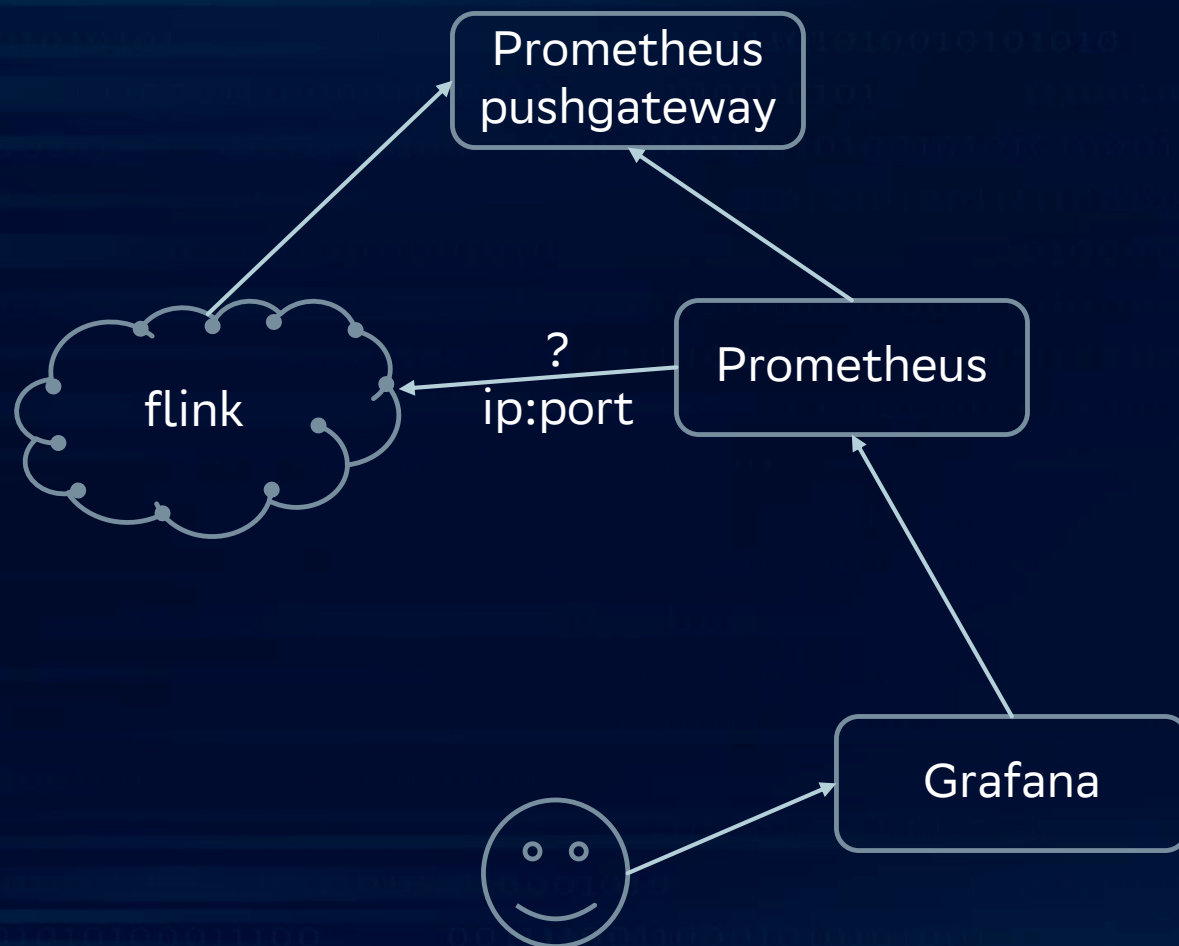
<https://github.com/apache/flink/blob/master/flink-dist/src/main/resources/config.yaml>

Monitoring

Rest API



Prometheus



Metrics, scope

<system scope> [+ <user scope>] + <metric name>

User scope

```
class CarCounter(MapFunction):
    def __init__(self):
        self.counter = None
    # Метод open вызывается один раз при инициализации каждого слота задачи
    def open(self, runtime_context: RuntimeContext):
        # Регистрация метрики
        # Группа создается автоматически или задается через .add_group()
        self.counter = runtime_context.get_metrics_group() \
            .add_group("MyCustomMetricsGroup") \
            .counter("carCounter")
    # Метод map вызывается для каждого элемента потока
    def map(self, value):
        # Инкремент метрики при получении каждой строки
        self.counter.inc()
        # Возвращаем значение без изменений (pass-through)
        return value
```

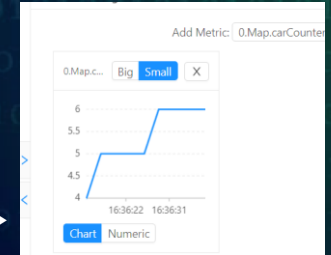
System scope

metrics.scope.jm
metrics.scope.jm-job
metrics.scope.tm
metrics.scope.tm-job
metrics.scope.task
metrics.scope.operator

format

<host>.taskmanager.<tm_id>.<job_name>.<operator_name>.<subtask_index>

localhost.taskmanager.21.Job2.Op10.0.metric



Failure Recovery

Checkpoint disable => default none

Checkpoint enable => default exp-delay(maxint, 1s)

Restart strategy

- none, off, disable
env.setRestartStrategy(RestartStrategies.noRestart());
- fixeddelay, fixed-delay
env.setRestartStrategy(RestartStrategies.fixedDelayRestart(10, Time.of(3, TimeUnit.SECONDS)));
- failurerate, failure-rate
restart-strategy.failure-rate.max-failures-per-interval: 3
restart-strategy.failure-rate.failure-rate-interval: 5 min
restart-strategy.failure-rate.delay: 10 s
- exponentialdelay, exponential-delay
restart-strategy.exponential-delay.initial-backoff: 3 s
restart-strategy.exponential-delay.max-backoff: 4 hour
restart-strategy.exponential-delay.backoff-multiplier: 2.0
restart-strategy.exponential-delay.reset-backoff-threshold: 30 min
restart-strategy.exponential-delay.jitter-factor: 0.1

Failover strategy

- full
- region

Runtime configuration

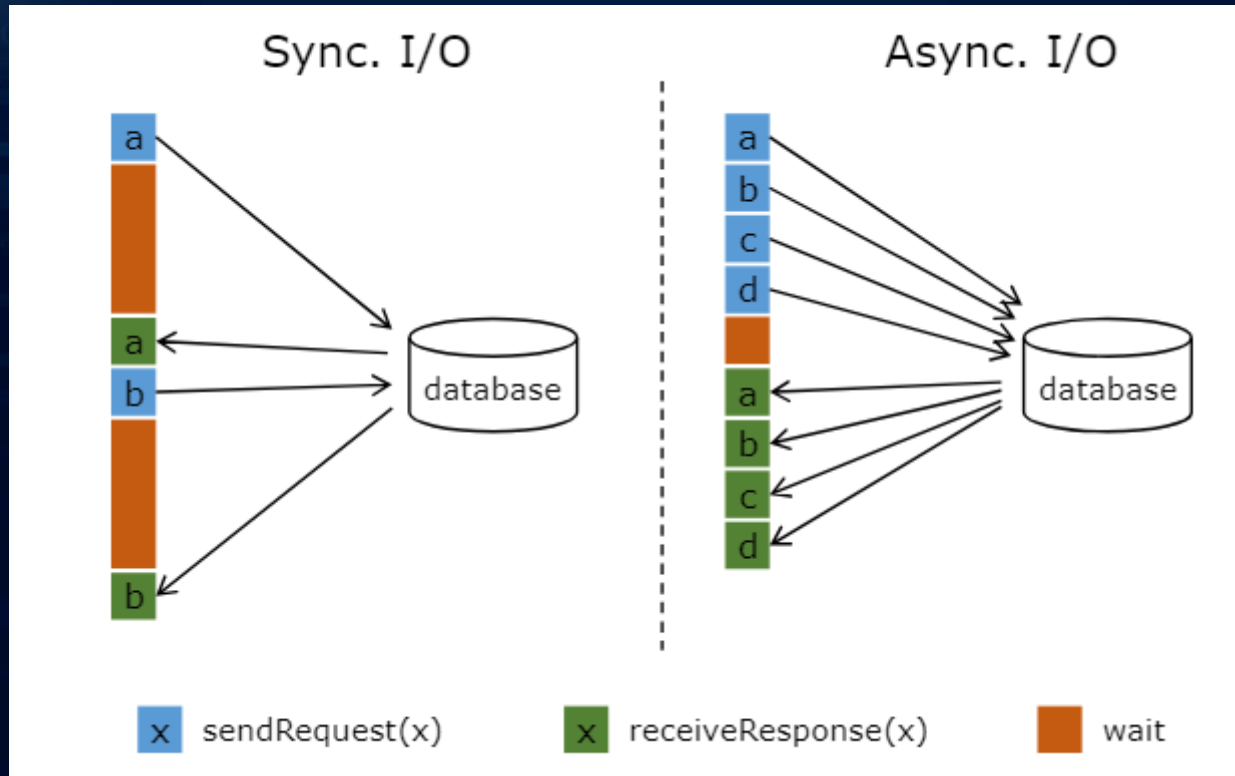
Job parallelism

Restart strategies

Serialization

ExecutionConfig visible via RuntimeContext

Async IO



Parameters

- Timeout
- Capacity
- AsyncRetryStrategy

Results order

- Unordered
- Ordered

Async IO != multithreading

Async IO requires support asynchronous requests

In Flink 2.2, we have added support of async function in Python DataStream API. This enables Python users to efficiently query external services in their Flink jobs, e.g. large-sized LLM which is typically deployed in a standalone GPU cluster, etc

<https://nightlies.apache.org/flink/flink-docs-master/docs/dev/datastream/operators/asyncio/>

<https://flink.apache.org/2025/12/04/apache-flink-2.2.0-advancing-real-time-data--ai-and-empowering-stream-processing-for-the-ai-era/>

Delivery guarantees

1. Нельзя терять данные?

Нет

#нет гарантий #no checkpoints

Да

2. Корректность результатов?

Нет

#воспроизводимые источники #at least once

Да

3. Отсутствие дублей?

Нет

#exactly once #воспроизводимые источники

Да

#exactly once #транзакционные
получатели #воспроизводимые
источники

At most once

At least once

Exactly once

Checkpointing

Disable checkpoints
all state lost
at most once

exactly once processing != at least once + idempotent operators

- +repeatable sources
- +event comes via state at least once

unalignment checkpoints

```
env.getCheckpointConfig().setCheckpointingMode(CheckpointingMode.AT_LEAST_ONCE);
```

- +event comes via state exactly once

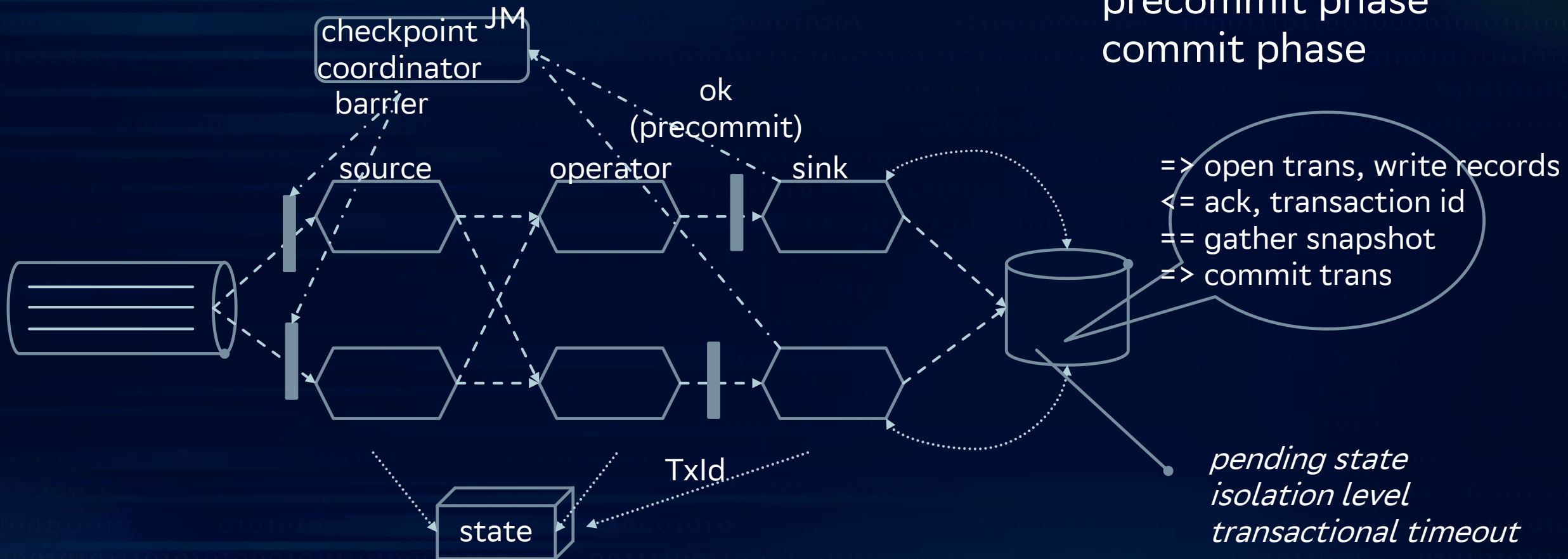
```
env.getCheckpointConfig().setCheckpointingMode(CheckpointingMode.EXACTLY_ONCE);
```

Exactly once, end to end

+transactional sinks

2PC

precommit phase
commit phase



Word count, delivery guarantee

+Checkpoint: disabled

= = некоторые слова не учтены, нет дублей

+Checkpoint: at least once

+repetable sources

= = некоторые слова подсчитаны несколько раз , возможны дубли в результатах

+Checkpoint: exactly once

+repetable sources

= = слова подсчитаны правильно, возможны дубли в результатах

+Checkpoint: exactly once

+repetable sources

+transactional sinks

= = слова подсчитаны правильно, нет дублей

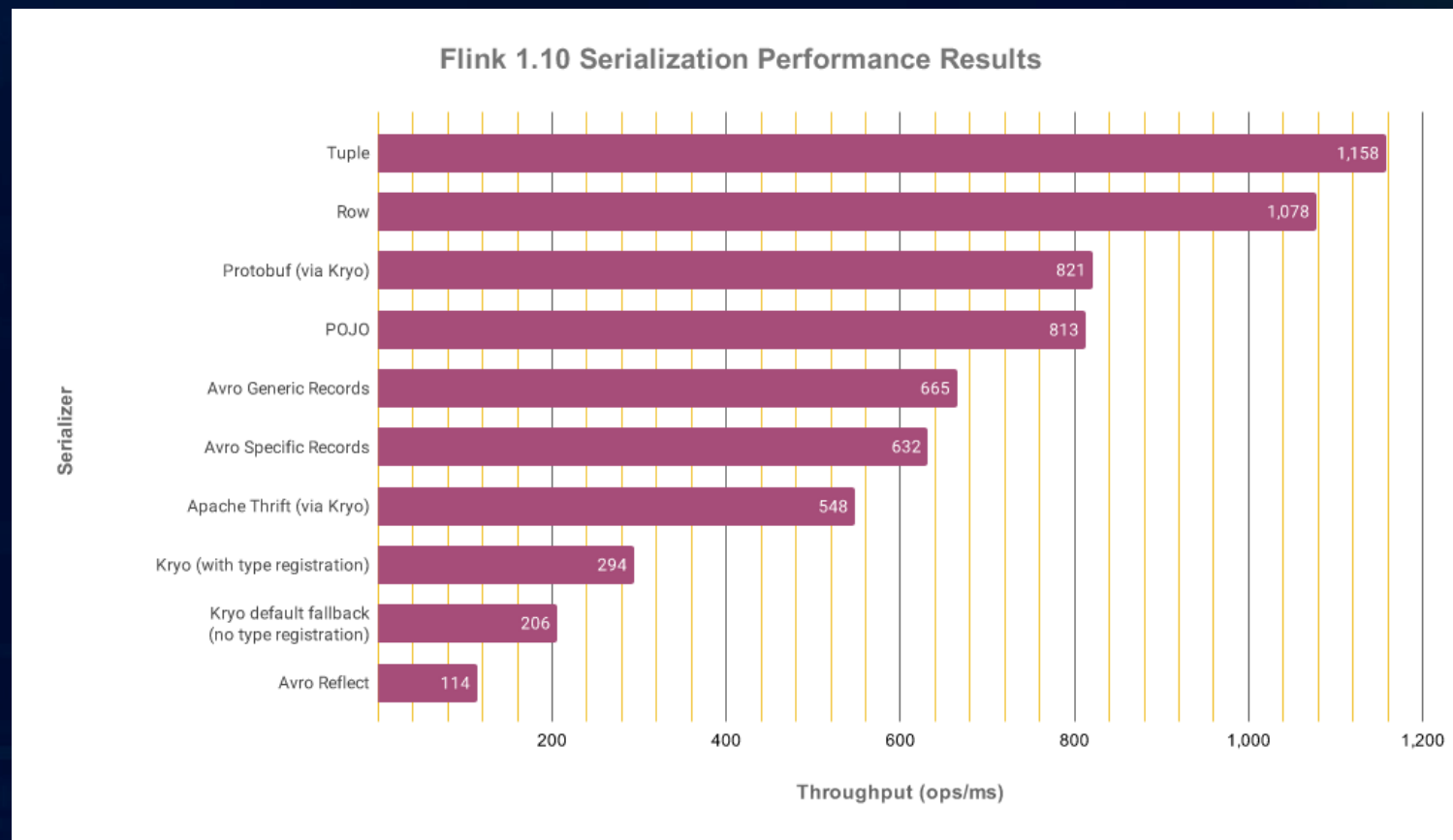
Serialization

Дорого (состояние, обмен сообщениями)

Выбрать правильную технику и формат

Подсчитать в граммах в байтах

Попробовать изменить тип данных, сериализатор, поток исполнения



<https://flink.apache.org/2020/04/15/flink-serialization-tuning-vol.-1-choosing-your-serializer-if-you-can/>

Schema registry

На примере формата avro

Схема данных поставляется внутри сообщения

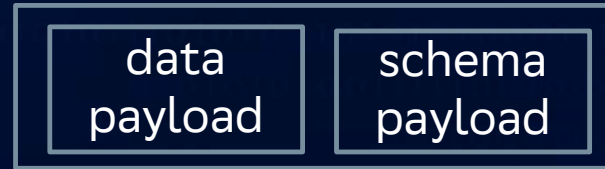
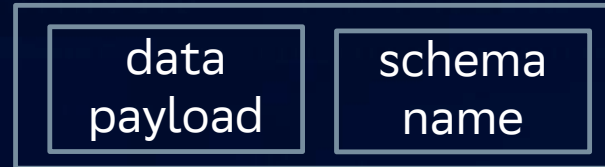
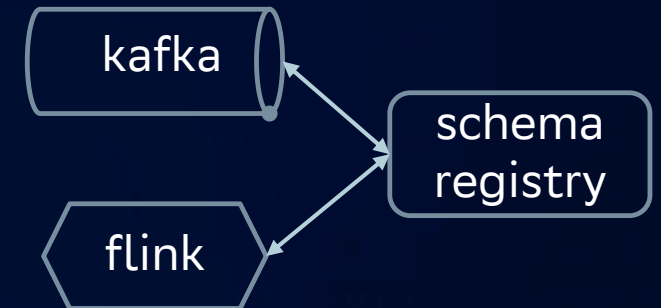
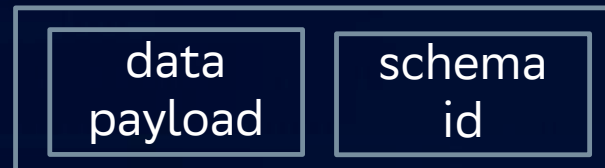


Схема поставляется в виде имени



Включен идентификатор схемы



Memory, process utilization

job & cluster configuration

throughput (network, kafka, cluster, etc)

Учет пиков

Знать свои задачи

мощность потока в rps

latency

state

gurances

recovery – checkpoint interval

message size

Example



Hardware: 10 BM
 Data: 10kb
 Throughput: 100k rps
 Checkpoint: 1m
 Keys: 10k distinct keys

Benchmarking!

local disk

state: window key 40bytes*10k keys / 60 sec = 7kb/s rw
 key access (r, upd, w): 40b*100krps/10TM=400kb/s rw

-kafka

kafka-keyby

keyby-groupby

groupby-kafka

kafka-

network

Per node

10kb*100k rps / 10 BM = 100mb/s
 in: 100mb/s
 out: 100mb/s

10kb*100k rps = 1gb/s
 1 gb/s / 10 BM = 100mb/s
 in: 100mb/s

100mb/s / 10TM = 10mb/s local
 10 mb/s * (10-1) = 90mb/s remote
 in: 90mb/s
 out: 90mb/s

10k keys*10kb/10/300 = 30kb/s
 out: 30kb/s

in: 30kb/s
 out: 30kb/s

disk total r: 407kb/s

disk total rw: 407kb/s

network total in: 290.3mb/s

network total out: 190.6mb/s

Monitoring

streaming != batch

memory consumption

state size

checkpointing (success/failed)

checkpoint duration

backpressure

latency,

event time lag

full latency

oper delay

throughput

jvm metrics

System cluster metrics

Builtin flink metrics

environment metrics

kafka lag

hbase, spark, etc

uptime

numRestarts

restartingTime

Alerting!

custom metrics!

Analysis



throughput & latency

resource utilization

delivery guarantees

scaling

recovery

downtime

evolution

adaptability (10 events per day)

custom metrics

event / process time

Predict

Workload last & historical (metrics, short and long term) ←



Understanding your limitations (analysis, infrastructure) ←



Predict workload (ml regression) and optimal cluster size ←



Scale up&down your tasks (ext autoscaling mechanics)



Do remember your maximum throughput per given infrastructure (parallelism) •

Some worst practices

Синхронизация потоков, упорядочивание сообщений

- Медленно
- Дорого
- Странно (баги, backpressure)

Использование тредов?

- Использовать `async io`
- Использовать `timers`
- Использовать параллелизм

Пакетная постановка задачи

- Уточнить требования
- Выбрать другой инструмент

DataStream API vs Table&SQL API

ProcessFunction vs Window

Сложные типы данных

- Потребление памяти, сети
- Нагрузка на процессор при сериализации
- Тяжелый ключ – медленная работа с состоянием

text & json vs avro & protobuf

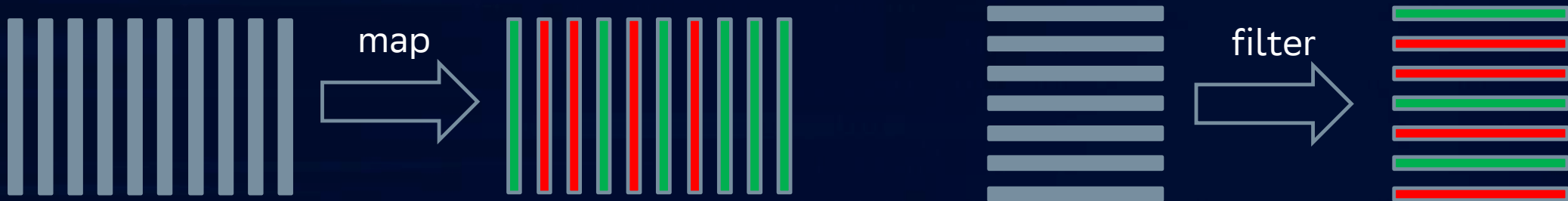
simple map & filter

```
ds.map().keyby().window().aggr().filter().sink()
```

Обрезать лишние поля

Отфильтровать лишние сообщения

Обрабатывать поток десериализую только нужные поля



Production Readiness Checklist

Set An Explicit Max Parallelism

Set UUIDs For All Operators

Choose The Right State Backend

- What is the SLA of your service

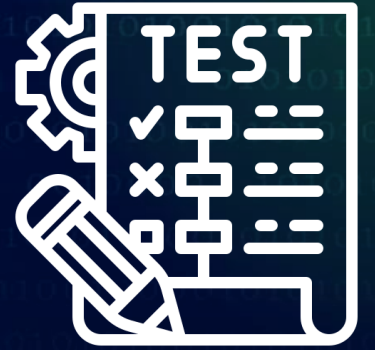
- How often must your service deliver results

- How much load can your Task Managers sustain

Configure JobManager High Availability

https://nightlies.apache.org/flink/flink-docs-release-1.18/docs/ops/production_ready/

Testing



Testing is an integral part of every software development process as such Apache Flink comes with tooling to test your application code on multiple levels of the testing pyramid.

<https://nightlies.apache.org/flink/flink-docs-master/docs/dev/datastream/testing/>

<https://github.com/apache/flink/tree/master/flink-streaming-java/src/test>

Community

<https://flink.apache.org/community/>

https://join.slack.com/t/apache-flink/shared_invite/zt-1thin01ch-tYuj6Zwu8qf0QsivHY0anw

<https://stackoverflow.com/>

user@flink.apache.org

Homework



Questions



Спасибо за внимание